

SUPPORT VECTOR MACHİNES

hem sınıflandırma hem de regresyon problemleri için kullanılan bir **denetimli makine öğrenmesi algoritmasıdır**. Başlıca amacı, farklı veri sınıfları arasında en iyi ayırma düzlemini (hiperdüzlem) bularak yeni verileri doğru bir şekilde sınıflandırmaktır.

Vektörel Gösterim $w^T x + b = 0$ Ne anlama geliyor?

$x \rightarrow$ **Özellik vektörü**. Bir **veri noktasını** temsil eden bir vektördür. Diğer bir deyişle, sınıflandırmaya çalıştığımız bir şeyin (örneğin bir hasta, bir e-posta, bir resim) tüm özelliklerinin bir listesidir.

*Örnek olarak, bir hastanın kalp hastalığı riskini tahmin etmek istediğimizi düşünelim. Bu durumda, bir hastanın x vektörü şunlardan oluşabilir: $x = [\text{yaş}, \text{kilo}, \text{kan basıncı}, \dots]$

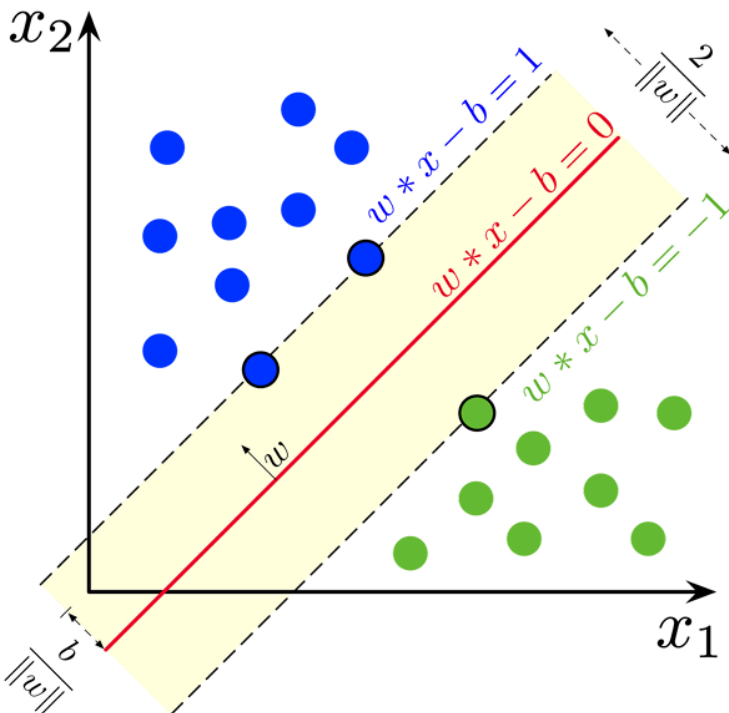
$w \rightarrow$ **Ağırlık Vektörü**. hiperdüzlemin eğimini ve yönünü belirleyen **ağırlık vektörüdür**. Her bir özelliğin sınıflandırma üzerindeki önemini gösterir. Bir nevi, modelin her bir özelliğe verdiği katsayıdır.

*Model, eğitim sırasında her bir özelliğin doğru sınıflandırma için ne kadar önemli olduğunu öğrenerek bu ağırlıkları belirler. Örneğin, eğer "kolesterol seviyesi" kalp hastalığı için en önemli göstergeyse, w vektöründeki "kolesterol_seviyesi"ne karşılık gelen değer diğerlerinden daha büyük olabilir

w^T —Ağırlıklar vektörünün transpozisi. Örneğin iki boyutlu bir problemde: $w^T = [w_1 \ w_2]$
 x ise iki boyutlu bir problemde 2×1 lik bir matris dir. $x = [[x_1][x_2]]$.

$$w^T x = w_1 x_1 + w_2 x_2$$

Support Vector Classifies (SVC)



SVC'nin ana hedefi, farklı veri sınıfları arasında en geniş boşluğu (marjı) yaratacak bir ayırma çizgisi veya düzlemi bulmaktır. Bu ayırma düzlemine **hiperdüzlem** denir.

SVC, bu hiperdüzlemi belirlerken, her iki sınıfa da en yakın olan veri noktalarına odaklanır. Bu kritik noktalara **destek vektörleri (support vectors)** denir. Algoritmanın adını veren bu vektörler, hiperdüzlemin konumunu ve yönünü belirler.

yukarıdaki resme odaklanacak olursak;

Hiperdüzlem (Kırmızı Çizgi): Bu, iki sınıfı birbirinden en iyi şekilde ayıran ana çizgidir. Denklem olarak $\mathbf{w} \cdot \mathbf{x} - \mathbf{b} = 0$ şeklinde ifade edilir. Bu çizgi, yeni bir veri noktasının hangi sınıfa ait olacağını belirleyen karar sınırır.

Destek Vektörleri (Siyah Çerçevesi Daireler): Hiperdüzleme en yakın olan veri noktalarıdır. Görselde bir mavi ve bir yeşil dairenin etrafı siyah bir halka ile çevrili. Bu noktalar, modelin konumunu ve yönünü belirleyen kritik noktalar.

Marj (Sarılı Alan): Destek vektörleri üzerinden geçen paralel kesikli çizgilerle (tüp benzeri) belirlenen boşluktur. Bu boşluğun boyutu, modelin ne kadar güvenilir olduğunu gösterir. **SVM'in amacı, bu marjı en geniş hale getirmektir.**

Marj Sınırları (Kesikli Çizgiler): Destek vektörleri üzerinden geçen ve hiperdüzleme paralel olan sınırlardır. Denklemleri $\mathbf{w} \cdot \mathbf{x} - \mathbf{b} = 1$ (mavi sınıf için) ve $\mathbf{w} \cdot \mathbf{x} - \mathbf{b} = -1$ (yeşil sınıf için) olarak gösterilir.

Marj Genişliği: Marjın genişliği, $2 / \|\mathbf{w}\|$ formülüyle hesaplanır. SVM, bu değeri maksimize etmek için $\mathbf{w}$ vektörünün normunu ($\|\mathbf{w}\|$) minimize etmeye çalışır.

SVM algoritması, aslında bir optimizasyon problemi çözer. Bu problem, belirli kısıtlamalar altında $\|\mathbf{w}\|$ değerini en aza indirmektir:

Hedef: $\|\mathbf{w}\|$ değerini minimize et.

*****SVC, bu hiperdüzlemi belirlerken, her iki sınıfa da en yakın olan veri noktalarına odaklanır. Bu kritik noktalara destek vektörleri (support vectors) denir. Algoritmanın adını veren bu vektörler, hiperdüzlemin konumunu ve yönünü belirler.**

*****SVC'nin amacı,** her zaman ikinci senaryoyu, yani **en geniş marjı** bulmaktır. Çünkü en geniş marj, modelin bilinmeyen verilere karşı en dayanıklı ve en güvenilir halidir. Dar bir marj, modelin "sadece bu verilere göre" çalıştığının bir işaretidir; geniş bir marj ise "genel olarak bu şekilde çalışırım" der.

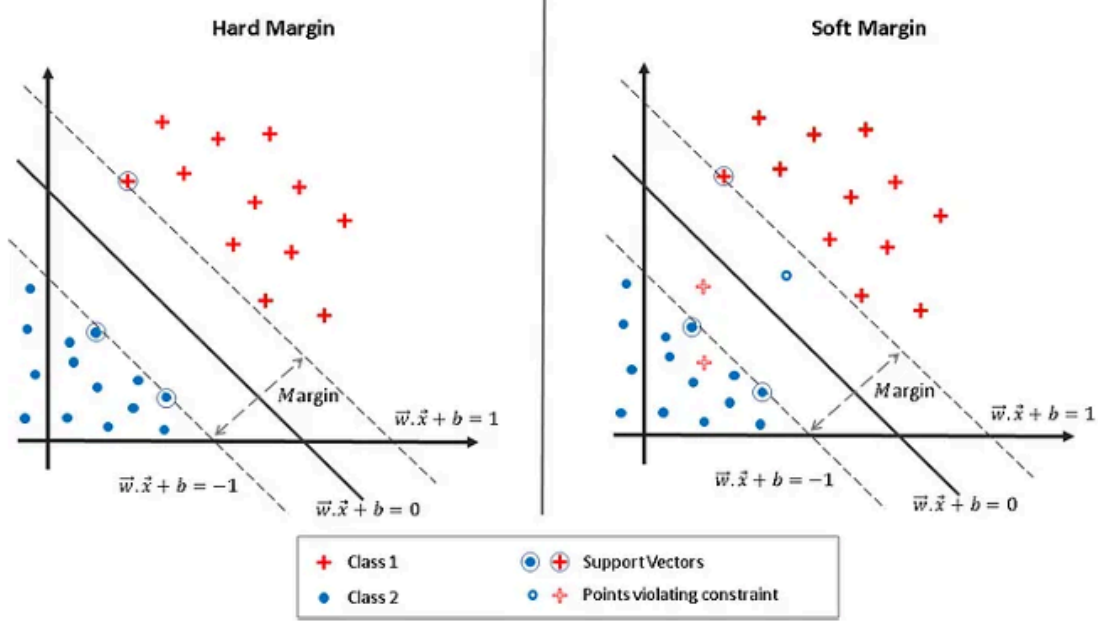
Yeni Veri Noktaları ve Marj

SVC modelini eğittikten sonra, yeni bir veri noktası geldiğinde (daha önce hiç görmediğimiz bir veri), model bu noktanın özelliklerine bakar ve onu hiperdüzlem denklemine yerleştirir.

- **Marjın Dışında:** Eğer yeni veri noktası, hiperdüzlemden destek vektörlerine olan mesafeden daha uzaktaysa, model o noktayı yüksek bir güvenle sınıflandırır. Bu, noktanın **marjın dışında** kaldığı anlamına gelir. Bu, modelin en çok tercih ettiği durumdur.
- **Marjın İçinde:** Eğer yeni veri noktası, marjın sınırları içinde kalırsa (yani, destek vektörlerine çok yakınsa), modelin bu noktayı sınıflandırma güveni daha düşüktür. Bu noktalar, iki sınıfın arasında kalan "gri alan" olarak görülebilir. Teorik olarak,

yumuşak marjlı bir modelde, bu noktalar da sınıflandırmaya dahil edilir, ancak modelin kararı daha belirsizdir.

Hard Margin(Katı Marj) ve Soft Margin(Yumuşak Marj)



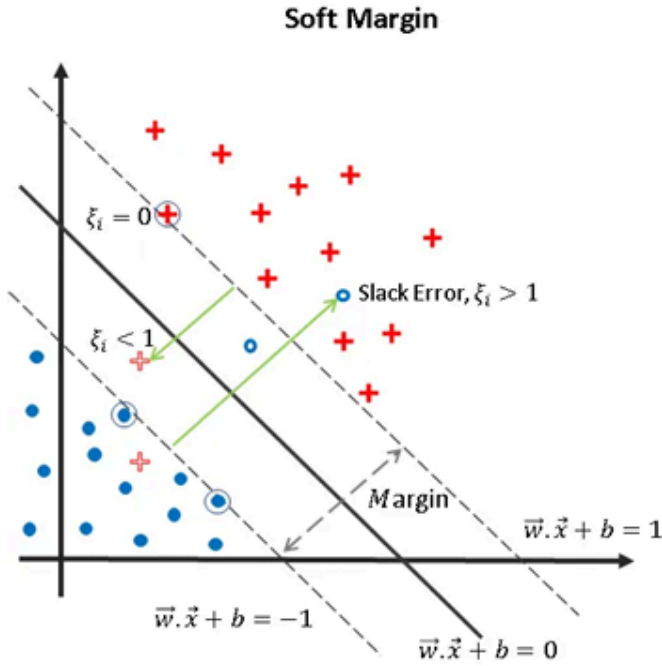
Hard Margin, en basit haliyle, bir modelin **hiçbir hataya izin vermeyen** yaklaşımıdır.

- **Şart:** Veri noktaları, bir çizgiyle (veya hiperdüzlemle) **mükemmel şekilde ayrılabilir** olmalıdır. Yani, bir sınıfın hiçbir elemanı diğer sınıfın içine sızmamalıdır.
- **Çalışma Prensipleri:** Model, iki sınıf arasına hiçbir veri noktasını içermeyen, en geniş boşluğu bulur. Tüm veri noktaları, hiperdüzlemin doğru tarafında kalmak zorundadır.
- **Ne Zaman Kullanılır?** Sadece veri setiniz temiz ve gürültüden arındırılmışsa kullanılır. Eğer veri noktaları birbirine karışmışsa, hard margin SVM bir çözüm bulamaz ve başarısız olur.

Soft Margin, hard margin'in gerçek dünya problemlerine uyarlanmış, **daha esnek ve hoşgörülü** bir versiyonudur.

- **Şart:** Veri setinin mükemmel şekilde ayrılabilir olması gerekmez. Veri noktalarının birbirine karışmasına veya aykırı değerlerin varlığına izin verir.
- **Çalışma Prensipleri:** Model, en geniş marjı bulmaya çalışır, ancak bunu yaparken bazı veri noktalarının yanlış tarafa düşmesine veya marjın içine girmesine göz yumar.
- **Kontrol Parametresi:** Bu hoşgörü, **Ceza Parametresi (C)** ile kontrol edilir.
 - **Büyük C:** Hatalara karşı çok serttir. Model, mümkün olduğunca az hataya izin verir ve dar bir marj oluşturarak veriye daha sıkı uyum sağlar (**aşırı öğrenme** riski taşır).

- **Küçük C:** Hatalara karşı daha toleranslıdır. Model, daha geniş bir marj oluşturur ve aykırı değerleri görmezden gelerek daha iyi genelleme yapar.



hinge-loss:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

C: ceza katsayısı

Amac bu hatayı minimize etmek!

ya w azalacak o zaman tüp genişler, ya da C ile hataların toplamını azaltcaz.

Figure 3: Suraj Donthi Soft Margin Görseli

xi (Xi - Hata Payı): Bu sembol, her bir veri noktasının hatasını (yani marjın dışına veya yanlış tarafa ne kadar taşıdığını) ölçer.

- $\xi_i=0$: Nokta, marjın doğru tarafında ve doğru konumdadır. Hata sıfırdır.
- $\xi_i<1$: Nokta, marjın yanlış tarafında ama hiperdüzleme çok yakındır. Hata payı birden küçüktür.
- $\xi_i>1$: Nokta, hem marjın hem de hiperdüzlemin yanlış tarafındadır. Hata payı birden büyüktür.

Eğer ki C çok yüksek olursa;

$$-C \sum_{i=1}^n \xi_i$$

bu değer sürekli artar değil mi. Biz genel hata formülünü değerini azaltmak için **slack-error(xi)** değerlerini azaltmaya çalışacağız. Ee bunun için ise **Marj'ın** destek vektörlerini biraz daha kısaltırız yani doğruya yaklaştırırız. **Marj'ı** küçültmüş oluruz.

Support Vector Regression (SVR)

SVM algoritmasının regresyon problemlerine uyarlanmış halidir. Sınıflandırma (SVC) bir şeyleri kategorilere ayırırken, regresyon (SVR) **sürekli bir değer tahmin etmeye** odaklanır. Örneğin, bir evin fiyatını, bir kişinin yaşını veya bir hisse senedinin değerini tahmin etmek için kullanılır.

*SVC, bir ayırma çizgisi bulurken, SVR **veriye en iyi uyan bir doğru veya eğri** bulur. Bu doğru veya eğri, tahmin modelinizi temsil eder.

–Ancak SVR'nin en büyük farkı, geleneksel regresyon modelleri gibi tüm veri noktalarının tahmin doğrusuna olan mesafesini minimuma indirmeye çalışmamasıdır. Bunun yerine, SVR bir **hata toleransı** belirler

ϵ (Epsilon) Duyarsızlık Bölgesi

SVR, tahmin çizgisinin etrafında bir ϵ (**epsilon**) **genişliğinde duyarsızlık bölgesi** oluşturur. Bu, SVR'nin en temel ve en önemli özelliğidir.

- **Bölge İçi Noktalar:** Tahmin çizgisinin **bu bölge içinde kalan tüm veri noktaları "hatasız" kabul edilir**. Model, bu noktalar için herhangi bir ceza vermez.
- **Bölge Dışı Noktalar:** Yalnızca **bu bölgenin dışında kalan veri noktaları için bir ceza uygulanır**.

Bu yaklaşım, modelin aşırı öğrenmeye karşı daha dayanıklı olmasını sağlar çünkü her veri noktasını kusursuz bir şekilde tahmin etmeye çalışmaz.

Aykırı Değerlere Karşı Dayanıklılık: ϵ duyarsızlık bölgesi sayesinde, model aykırı değerlerden (outliers) daha az etkilenir. Geleneksel regresyon modelleri, aykırı değerlere uyum sağlamak için tahmin çizgisini çarpıtabilirken, SVR bu tür noktaları "bölgenin dışında" olarak ele alır ve sadece onları cezalandırarak modelin genel yapısını korur.

Eğitim Aşaması (Modeli Öğrenme):

- SVR, elinizdeki eğitim verilerini inceler.
- Amacı, verilerin çoğunu bu tünelin içine alacak en iyi doğruyu bulmaktır.
- Tünelin dışına çıkan noktalara ceza uygular. Bu cezayı, **C** parametresiyle belirlersiniz.
- Tüm noktalar bu tünelin içine giremeyebilir, ancak SVR sadece dışarıda kalanları umursar. En az sayıda noktayı dışarıda bırakacak şekilde tünelin konumunu ayarlar.

Ceza Verme Olayı ve Sonuçları

SVR'de ceza, modelin hatayı ne kadar ciddiye alacağını belirler. Bu ceza, **C parametresi** tarafından yönetilir.

- **C Parametresinin Rolü:** C bir hiperparametredir, yani sizin belirlemeniz gereken bir değerdir. C'yi ne kadar yüksek tutarsanız, model hata yaptığında o kadar büyük bir "ceza" alır.
- **Ceza Verilince Ne Olur?**
 - Eğer bir veri noktası **epsilon tüpünün dışına** düşerse, model o nokta için bir hata payı (slack variable) hesaplar ve bu hata payını C ile çarparak bir ceza puanı elde eder.
 - Modelin optimizasyon problemi, bu ceza puanını minimuma indirmeyi hedefler.
 - **Büyük C değeri** kullanırsanız, model tüpün dışındaki noktalara çok sert bir ceza verir. Bu, modelin mümkün olduğunca az noktayı tüpün dışında bırakmaya çalışmasına neden olur. Sonuç olarak, tüp daha daralır ve model eğitim verisine daha fazla uyum sağlar, ancak bu durum **aşırı öğrenmeye (overfitting)** yol açabilir.
 - **Küçük C değeri** kullanırsanız, model hatalara daha toleranslı olur. Tüpün dışına çıkan noktalar için daha az ceza alır, bu yüzden model daha esnek bir tüp oluşturur ve genelleme yeteneği artar.

Neden Tüp Var? Geleneksel regresyon (örneğin, Lineer Regresyon) her bir veri noktasının doğruya olan mesafesini minimuma indirmeye çalışır. Bu, aykırı değerler geldiğinde doğrunun o noktaya doğru kaymasına ve tüm modelin bozulmasına neden olabilir. SVR'nin tüpü, bu tür bozulmaları engelleyen bir güvenlik yastığı gibidir.

Doğru Neye Göre Çiziliyor?

SVR'deki doğru veya eğri, bir optimizasyon problemi çözülerek çizilir. Bu problemin amacı, iki ana koşulu sağlamaktır:

1. **En Düz Çizgiyi Bulmak:** Mümkün olan en az karmaşıklığa sahip doğruyu veya eğriyi bulmak. Matematiksel olarak bu, ağırlık vektörü **w'nin normunu ($\|w\|$) minimize etmektir**. Düşük $\|w\|$ değeri, daha düz (less complex) bir doğru anlamına gelir.
2. **Hata Puanını Minimize Etmek:** Tüpün dışına düşen noktaların neden olduğu toplam ceza puanını en aza indirmek. Bu, C parametresi ve hata payları (ξ ve ξ^*) ile birlikte optimize edilir.

Model, bu iki hedef arasında bir denge kurmaya çalışır: **Hem en düz çizgiyi bulmak hem de tüpün dışındaki noktaların cezasını en aza indirmek**. Bu optimizasyon süreci sonucunda, modelimiz için en ideal tahmin doğrusu (veya eğrisi) belirlenmiş olur.

***O meşhur "tüp" kavramı, sadece modelin bu doğruyu nasıl oluşturacağını belirleyen bir araçtır. Eğitim aşaması tamamlandığında, tüpün içindeki ve dışındaki tüm noktaların konumu belirlenmiş olur ve model artık sadece o tek doğruyu kullanarak tahmin yapar.

Neden Lineer Regresyon Varken SVR Kullanırız?

Bu sorunun cevabı, SVR'nin Lineer Regresyon'a göre sunduğu temel avantajlarda yatar:

- **Aykırı Değerlere Karşı Sağlamlık:** Lineer regresyon, aykırı bir veri noktasından ciddi şekilde etkilenir ve tüm doğruyu o noktaya doğru eğme eğilimi gösterir. Bu, modelin genelleme yeteneğini düşürür. SVR ise aykırı değerleri görmezden gelmek için tüpü kullanır ve modelin ana yapısını korur.
- **Doğrusal Olmayan Verileri Modelleme:** Lineer regresyon sadece doğrusal ilişkileri modelleyebilir. Eğer veri noktalarınız bir eğri oluşturuyorsa, Lineer Regresyon başarısız olur. SVR ise **kernel hilesi** sayesinde doğrusal olmayan ilişkileri bile modelleyebilir. Bu, SVR'yi çok daha esnek ve güçlü bir araç haline getirir.
- **Karmaşıklık Kontrolü:** SVR'de **C** ve **epsilon** gibi hiperparametreler sayesinde modelin karmaşıklığını ve hata toleransını hassas bir şekilde ayarlayabiliriz. Bu, aşırı öğrenme (overfitting) ve eksik öğrenme (underfitting) riskini yönetmemize yardımcı olur.

SVR'nin "Sadece En Önemli Noktalara Odaklanma" Mantığı

Bir grup insanın boy ortalamasını bulmaya çalıştığınızı düşünün.

Lineer Regresyon gibi düşünürseniz: Herkesin boyunu tek tek ölçüp, ortalamayı bulmak için tüm değerleri toplar, sonra da kişi sayısına bölersiniz. Herkesin boyu bu ortalamayı etkiler.

SVR gibi düşünürseniz: Daha akıllı bir yöntem kullanırsınız. Herkesin boyunu ölçmek yerine, "toplumun geneli 1.65 cm ile 1.85 cm arasında" gibi bir kural koyarsınız. Bu aralık, SVR'nin **"tüpü"** olur.

- **Tüpün İçindeki Herkes:** Bu aralıkta (1.65-1.85 cm) olan herkesi "normal" kabul eder ve onların boyunu dikkate almazsınız. Çünkü onlar genel ortalamayı çok fazla değiştirmeyecektir.
- **Sadece Kritik Noktalar:** Sadece bu aralığın sınırında olan kişilere (1.65 cm ve 1.85 cm olanlar) ve bu aralığın dışına çıkan çok uzun veya çok kısa kişilere bakarsınız. İşte bu kişiler, **destek vektörleridir**.

Kernel Hilesi (Kernel Trick) ve Doğrusal Olmayan Ayırımlar

Kernel hilesi, SVM'in **doğrusal olmayan** (non-linear) problemleri çözmesini sağlayan güçlü bir tekniktir. Kısaca, veriyi daha yüksek boyutlu bir uzaya taşıyarak orada doğrusal bir ayırım bulmaya yarar.

Kernel hilesi, veriyi doğrudan işlemek yerine, aralarındaki benzerliği (ilişkiyi) hesaplayan bir fonksiyon (kernel) kullanır. Bu fonksiyon, veriyi otomatik olarak **daha yüksek boyutlu** bir uzaya taşır.

Bu sayede, SVM karmaşık ve doğrusal olmayan ilişkileri dahi, "düz bir çizgi" bulma mantığından vazgeçmeden çözebilir. En popüler kernel tipleri arasında **Polinom Kernel** ve **RBF (Radial Basis Function) Kernel** bulunur. Özellikle RBF, çok karmaşık veri setlerinde bile etkili bir çözüm sunar.

RBF için;

Çok Yönlüdür: RBF, çoğu doğrusal olmayan veri setinde iyi sonuç verir. Veri setinizle ilgili önceden bir bilginiz yoksa, genellikle RBF Kernel ile başlamanız önerilir.

Daha Az Ayar Gerektirir: Polinom Kernel gibi diğer kernel'lara göre daha az ayarlanabilir parametreye sahiptir, bu da kullanımını kolaylaştırır.

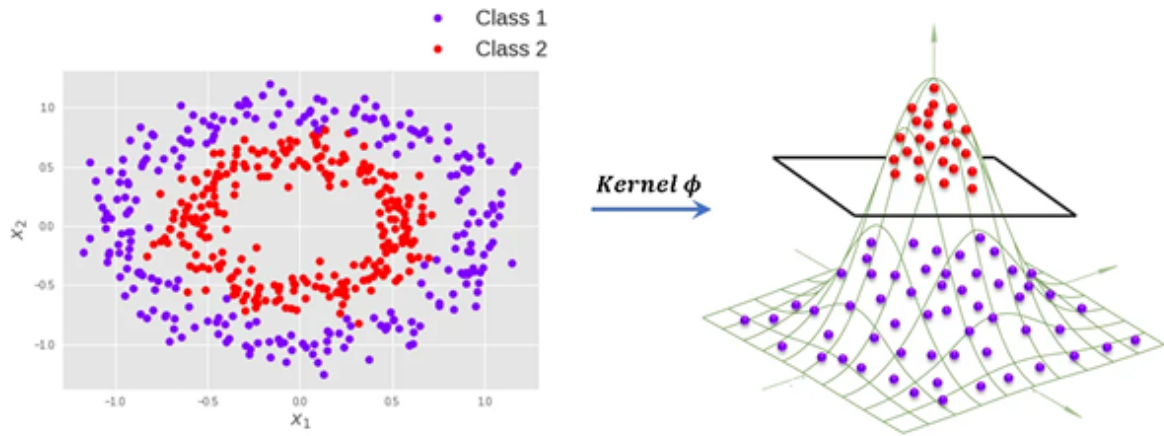


Figure 5: Suraj Donthi Kernel Görseli